

Amlogic_NPU_LiteRT指导

Revision: 0.1

Release Date: 2026-06-25

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称“Amlogic”）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址：www.amlogic.cn
- 售前咨询：contact@amlogic.com
- 技术支持：support@amlogic.com

修订记录

Revision	Change Description
v0.1 (2026-06-25)	初始版本

目录

1. LiteRT 概述

- 1.1 简介
- 1.2 主要特性
- 1.3 Amlogic 与 LiteRT 集成架构
- 1.4 Interpreter API
- 1.5 CompiledModel API
- 1.6 Interpreter 与 CompiledModel 对比
- 1.7 AML Delegate 简介

2. 平台部署总览

- 2.1 支持平台与交付形态
- 2.2 API 与平台选型
 - 2.2.1 平台选型
 - 2.2.2 API 选型
- 2.3 部署前检查
 - 2.3.1 硬件与版本
 - 2.3.2 算子与数据类型
 - 2.3.3 部署前清单
- 2.4 Demo 与示例索引

3. Android 部署

- 3.1 简介
- 3.2 支持情况
- 3.3 Interpreter API部署流程
 - 3.3.1 导入 LiteRT AAR
 - 3.3.2 编辑 app/build.gradle
 - 3.3.3 初始化 Interpreter 并挂载 AML Delegate
 - 3.3.4 设备侧调试 (Interpreter 本地可执行用例)
 - 3.3.5 Demo 与完整工程
- 3.4 CompiledModel API 部署流程
 - 3.4.1 Gradle 依赖
 - 3.4.2 Kotlin 快速入门
 - 3.4.3 与 Interpreter 的差异 (概要)
 - 3.4.4 设备侧额外库 (示例)
 - 3.4.5 Demo 与完整工程 (TODO)

4. Linux 部署

- 4.1 简介
- 4.2 支持情况
- 4.3 Interpreter API 部署
 - 4.3.1 Interpreter C API接口说明
 - 4.3.2 部署流程
 - 4.3.3 应用工程集成 (C/C++)
 - 4.3.4 验证建议
- 4.4 CompiledModel API 部署(TODO)

1. LiteRT 概述

LiteRT（Lite Runtime，TensorFlow Lite 的继任者）是 Google 面向端侧 AI 的统一推理框架。本文档在 [Google LiteRT 官方文档](#) 基础上，归纳 **Android 与 Linux 相关的集成与部署信息**，并说明 Amlogic **AML Delegate** 如何接入 LiteRT 以使用 ADLA NPU。

官方资源	链接
LiteRT 总览	developers.google.com/edge/litert
Android 开发	developers.google.com/edge/litert/android
示例应用	LiteRT Sample Apps on GitHub

1.1 简介

LiteRT 基于 TensorFlow Lite 演进而来，面向 Android、Linux、嵌入式等端侧平台，提供模型加载、硬件加速与推理执行能力。应用侧以 `.tflite` 为统一模型载体，可通过 **Delegate** 将子图下沉到 GPU / NPU 等加速器。

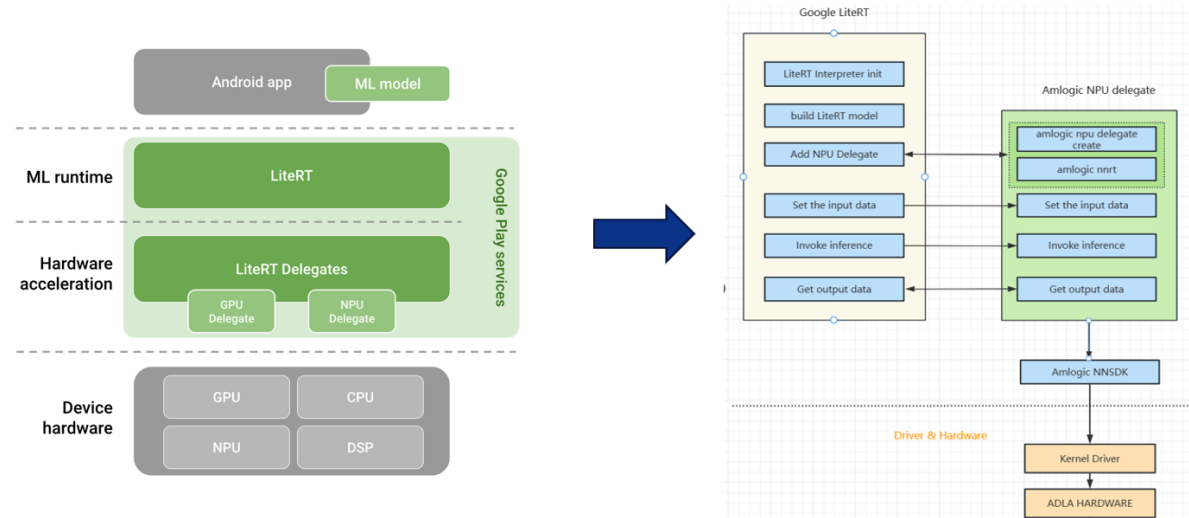
在 Amlogic 平台上，本文档重点说明如何将 LiteRT 与 **AML Delegate** 结合，在 Android APK 或 Linux Native 服务中完成 NPU 推理部署。

1.2 主要特性

- **端侧优化**：低延迟、隐私保护、离线可用、体积小、功耗低
- **跨平台**：支持 Android、Linux（含 Yocto / Debian 等嵌入式发行版）、iOS 等
- **多框架模型**：可将 TensorFlow、PyTorch、JAX 等模型转换为 `.tflite` 后运行
- **多语言 SDK**：Java/Kotlin、C++、Python 等
- **硬件加速**：通过 Delegate 或 CompiledModel 加速器选项对接 CPU / GPU / NPU

1.3 Amlogic 与 LiteRT 集成架构

下图展示 LiteRT 运行时与 Amlogic AML Delegate、NNSDK、ADLA 等模块的集成关系（待补充）：



1.4 Interpreter API

Interpreter API 归属于 LiteRT **V1.x** 系列（如：v1.2.0、v1.4.2），由 Google 官方维护，作为**基础推理层**，保证对原有 TFLite 及 LiteRT 应用的向后兼容。

项目	说明
官方定位	Basic inference API, maintained for backward compatibility
模型格式	<code>.tflite</code> (FlatBuffers)
硬件加速	通过 Delegate 机制（ <code>adddelegate</code> / <code>ModifyGraphWithDelegate</code> ）接入，支持 CPU / GPU / NPU 等后端（例如 AML Delegate）
典型用法	加载模型 → 配置输入/输出张量 → （可选）注册 Delegate → 调用 <code>run()</code> 或 <code>runForMultipleInputsOutputs()</code> 执行同步推理

Amlogic 调用链路：

```
应用 → LiteRT Interpreter → AML Delegate → Runtime → NNSDK → ADLA Driver → NPU
```

1.5 CompiledModel API

CompiledModel API 归属于 LiteRT **V2.x 系列**（如：v2.1.0、v2.1.5），由 Google 定义为**新一代高性能推理标准**。该 API 统一了 CPU / GPU / NPU 硬件加速路径，并提供异步执行能力及简化的设备选择机制。

项目	说明
官方定位	Modern standard for high-performance inference
模型格式	<code>.tflite</code>
硬件加速	通过 <code>CompiledModel.Options(Accelerator.CPU/GPU/NPU)</code> 显式指定，也支持厂商自定义扩展后端
典型用法	创建 <code>CompiledModel</code> 实例 → 分配输入/输出缓冲区（ <code>createInputBuffers / createOutputBuffers</code> ）→ 填充数据 → 执行推理（ <code>run</code> ）

Amlogic 调用链路：

```
应用 → LiteRT CompiledModel → Dispatch / Compiler Plugin → Runtime → NNSDK → ADLA Driver → NPU
```

1.6 Interpreter 与 CompiledModel 对比

对比项	Interpreter	CompiledModel
代表版本	v1.2.0 ~ v1.4.2 (Latest: v1.4.2)	v2.0.3 ~ v2.1.5 (Latest: v2.1.5)
核心 API	<code>Interpreter</code> + <code>addDelegate</code>	<code>CompiledModel.create</code> + <code>run</code>
NPU 接入	AML Delegate (<code>AmLDelegate</code>)	Dispatch / Compiler Plugin + AML 扩展
向后兼容	兼容既有 TFLite 代码	新 API，需按 V2 范式改造
Amlogic 成熟度	当前主交付路径	能力随版本演进
Google 建议	存量应用、快速迁移	新项目优先

版本与支持 API 对照表详见 [Google LiteRT Android 版本说明](#)。平台与 API 选型建议见 2.2。

1.7 AML Delegate 简介

LiteRT 模型采用 FlatBuffers 格式（`.tflite`）存储。推理过程中，**Delegate（委托）** 负责将可加速的计算子图**下放**至 GPU、DSP、NPU 等硬件后端执行，未被委托的部分则由 CPU 完成。

AML Delegate 是 Amlogic 为 **ADLA NPU** 提供的 LiteRT 硬件委托实现，它将 LiteRT 划分的可委托子图**转发**至 NNSDK / ADLA 运行时进行加速。该委托的硬件约束、算子支持及版本配套要求详见第 2.3 节。

2. 平台部署总览

2.1 支持平台与交付形态

平台	典型场景	交付形态
Android	APK / 系统应用、Java 业务层集成	Maven <code>com.google.ai.edge.litert</code> + <code>litert_aml_delegate.aar</code>
Linux	嵌入式服务、命令行/守护进程验证	<code>libLitert.so</code> + AML Delegate <code>.so</code> + 本执行用例；将 LiteRT / Delegate 库可编入镜像或推送到目标文件系统

本文档按**目标操作系统**划分为 Android 和 Linux 两套部署路径。两者在 **Interpreter + AML Delegate** 层面共享相同的 NPU 加速能力，差异主要体现在集成接口与交付物形态上：

对比项	Android 部署	Linux 部署
应用语言	Java / Kotlin（可辅以 JNI）	C / C++
Interpreter	<code>litert_aml_delegate.aar</code> + Maven <code>litert</code> ; AMLDelegate + Interpreter	<code>libLitert.so</code> + AML Delegate 相关库； Interpreter C API ； <code>aml_delegate_test_bin</code>
CompiledModel	额外集成 Dispatch / Compiler Plugin 等动态库	<code>libLitertDispatchAml.so</code> 、 <code>libLitertCompilerPluginAml.so</code> ； <code>aml_CompiledModel_benchmark</code>
模型部署路径	<code>assets</code> 、应用私有目录或指定绝对路径	文件系统绝对路径（如 <code>/data/...</code> 、工作目录）

Android 侧以 **AAR** 交付 AML Delegate；Linux 侧以 **共享库 + 可执行用例** 为主。

2.2 API 与平台选型

2.2.1 平台选型

- **选 Android**：业务在 APK 内、需 Java/Kotlin 集成、或需与 `com.google.ai.edge.litert` 官方依赖协同
- **选 Linux**：无 Android 框架、以 C/C++ 服务/命令行为主、或需在 bring-up / CI 上用本地 case 验证 NPU 链路

2.2.2 API 选型

根据项目阶段选择合适的推理 API：

- 已有 TFLite / Interpreter 工程，需快速接入 NPU → 优先选用 **Interpreter API + AML Delegate**
- 新建项目，需统一 CPU/GPU/NPU 加速路径并支持异步推理 → 评估并选用 **CompiledModel**

无论选择哪种 API，均需确保 **LiteRT 版本、AML Delegate / AAR 版本与 ROM / BSP 版本**保持一致

2.3 部署前检查

2.3.1 硬件与版本

项目	说明
硬件	需设备搭载 ADLA NPU ，且 ROM/BSP 中 ADLA 驱动可用
版本矩阵	LiteRT 版本、AML Delegate 版本、ROM/BSP/驱动版本 需配套
平台精度	Float32 / 量化模型支持因平台而异，详见06_Amlogic_NPU_Android_Deployment_Guide.pdf 1.4.2 章节

2.3.2 算子与数据类型

类别	支持说明
算子 (Op)	参考算子列表 support list.xlsx
浮点	Float32、Float16 等；需视端侧平台与模型配置
整型/量化	Int8、UInt8 等；支持 per-layer、per-channel 等常见量化方式
回退策略	无法由 NPU 执行的子图可由 LiteRT 回退至 CPU 执行，保证整图可跑

2.3.3 部署前清单

- 1. 确认目标板侧 **ADLA NPU** 与驱动正常；
- 2. 准备符合 TFLite 规范的 `.tflite`；
- 3. 对齐 **LiteRT / AML Delegate / BSP** 版本组合；
- 4. 规划模型路径与（若启用）**编译/加载缓存**目录，避免旧缓存与新模型混用。

2.4 Demo 与示例索引

API	平台	说明	文档章节
Interpreter	Android	最小 <code>Interpreter</code> + <code>AmlDelegate</code> 示例	3.4 （待补充完整 GitHub 链接）
Interpreter	Linux	<code>aml_delegate_test_bin</code> 本地 case	3.4.4、4.4
CompiledModel	Android	Kotlin / C++ 推理示例	3.5
CompiledModel	Linux	<code>main_npu</code> 等 <code>CompiledModel</code> case	4.5 (TODO)
官方参考	—	LiteRT Sample Apps	Google Android 文档

3. Android 部署

本章描述 Android 侧 **LiteRT Interpreter API** 与 **CompiledModel API** 的集成方式。

3.1 简介

本节介绍如何在 Android 应用中使用 LiteRT 与 **AML Delegate** 完成 APK 侧 NPU 推理。

- **Interpreter API**: `Interpreter` + `AmlDelegate` (见 3.3)
- **CompiledModel API**: `CompiledModel`

应用在 APK 中调用 LiteRT, 由 LiteRT 解析 `.tflite` 后, 通过 **AML Delegate** 将子图下放至 ADLA NPU。

3.2 支持情况

当前仅适配内置ADLA NPU的Amlogic系列设备, 算子覆盖范围、支持的数据类型与精度约束可查阅本文档2.3.2章节; 平台原生Float32运算与量化推理的性能、效果差异, 可参考配套文档《06_Amlogic_NPU_Android_Deployment_Guide.pdf》获取完整说明。

3.3 Interpreter API部署流程

3.3.1 导入 LiteRT AAR

将 `litert_aml_delegate.aar` 拷贝至应用源码目录下的 `libs/` 文件夹 (若不存在则手动创建)。

3.3.2 编辑 app/build.gradle

编辑 `app/build.gradle` 在 `dependencies` 块中引入本地 AAR 及 LiteRT 官方依赖。

```
dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar', '*.aar'])
    implementation 'com.google.ai.edge.litert:litert:1.2.0'
    implementation 'com.google.ai.edge.litert:litert-support-api:1.2.0'
}
```

3.3.3 初始化 Interpreter 并挂载 AML Delegate

- 导入 `AmlDelegate` 类

```
import org.tensorflow.lite.AmlDelegate;
```

- 创建 `AmlDelegate` 实例

```
private static AmlDelegate amlDelegate = null;
AmlDelegate.Options options_aml = new AmlDelegate.Options();
options_aml.model_path = ""; //双引号中填写板侧放置tflite模型的目录, 比如"/storage/emulated/0/DCIM/yolo_int8.tflite";
amlDelegate = new AmlDelegate(options_aml);
```

- 注册 `Delegate` 并构建 `Interpreter`

```
Interpreter.Options options = new Interpreter.Options();
options.addDelegate(amlDelegate);
private Interpreter tflite;
MappedByteBuffer modelFile = loadModelFile(context.getAssets(), modelFilename);
tflite = new Interpreter(modelFile, options);
```

- 加载模型的工具方法 (`loadModelFile`)

```
private static MappedByteBuffer loadModelFile(AssetManager assets, String modelFilename)
    throws IOException {
    AssetFileDescriptor fileDescriptor = assets.openFd(modelFilename);
    FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
    FileChannel fileChannel = inputStream.getChannel();
    long startOffset = fileDescriptor.getStartOffset();
    long declaredLength = fileDescriptor.getDeclaredLength();
    return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
}
```

- 执行推理

```
tfLite.runForMultipleInputsOutputs(inputArray, outputs);
```

3.3.4 设备侧调试（Interpreter 本地可执行用例）

当构建产物就绪后，可直接使用 **Interpreter 本地 case** 在设备上验证 NPU 链路（无需重新编译应用工程）：

- 本地 case: `aml_delegate_test_bin`
- 运行时依赖: `libLitedt.so` 及 AML Delegate 配套共享库（以实际发布包为准）

推送并执行：

```
adb push <path>/aml_delegate_test_bin /data/local/tmp/  
adb shell  
export LD_LIBRARY_PATH=/vendor/lib64:$LD_LIBRARY_PATH  
chmod +x /data/local/tmp/aml_delegate_test_bin  
/data/local/tmp/aml_delegate_test_bin <args>
```

3.3.5 Demo 与完整工程

项目	说明
示例内容	使用 <code>Interpreter</code> 加载 <code>.tflite</code> 模型、挂载 <code>AmlDelegate</code> 并执行推理
aar package Release	aar_package_link
完整工程	https://github.com/Amlogic-NN/amlnn-google-ai-ecosystem/tree/main/litert

3.4 CompiledModel API 部署流程

CompiledModel API 属于 LiteRT V2.x 系列（例如 v2.1.0），提供统一 CPU / GPU / NPU 加速路径及异步执行能力。当前多数 APK 场景仍以 Interpreter 流程（3.3 节）为主，CompiledModel 适用于需要统一多后端加速或异步推理的新项目。

3.4.1 Gradle 依赖

在 `app/build.gradle` 中引入 LiteRT V2.x 依赖：

```
dependencies {  
    implementation 'com.google.ai.edge.litert:litert:2.1.0'  
}
```

3.4.2 Kotlin 快速入门

以下示例摘自 Google 官方文档，演示加载模型、分配缓冲区并执行推理：

```
val compiledModel = CompiledModel.create(  
    "/path/to/mymodel.tflite",  
    CompiledModel.Options(Accelerator.CPU)) // 可替换为 GPU / NPU  
  
val inputBuffers = compiledModel.createInputBuffers()  
val outputBuffers = compiledModel.createOutputBuffers()  
  
inputBuffers.get(0).writeFloat(input0)  
compiledModel.run(inputBuffers, outputBuffers)  
val output = outputBuffers.get(0).readFloat()
```

3.4.3 与 Interpreter 的差异（概要）

项目	Interpreter（主路径）	CompiledModel（可选）
入口 API	<code>Interpreter</code> + <code>AmlDelegate</code>	<code>CompiledModel</code> 及配套编译/加载接口
额外依赖	通常包含在 <code>litert_aml_delegate.aar</code> 内	需要额外 Dispatch / Compiler Plugin 动态库

3.4.4 设备侧额外库（示例）

使用 CompiledModel 时，需将配套动态库推送到设备（具体库名以实际发布包为准）：

```
adb push <path>/libLiteRT.so /data/LiteRT/  
adb push <path>/libLiteRTDispatch_Aml.so /data/LiteRT/  
adb push <path>/libLiteRTCompilerPlugin_Aml.so /data/LiteRT/  
adb push <path>/aml_CompiledModel_benchmark /data/LiteRT/
```

具体运行结果见《06_Amlogic_NPU_Android_Deployment_Guide.pdf》

3.4.5 Demo 与完整工程（TODO）

项目	说明
示例类型	CompiledModel 加载 .tflite 模型、选择 Accelerator 并执行推理
官方参考	Image segmentation Kotlin/C++ Demo
aar package Release	(TODO)
完整工程	GitHub 链接 (TODO)

具体 API 调用与模型产物格式以对应版本发布说明为准；

4. Linux 部署

本章描述 Linux 侧 **Interpreter C API** 与 **CompiledModel** 的集成方式。

4.1 简介

Linux 侧不经过 Android Framework，采用 **C/C++ + 动态库 + 本地可执行用例** 的方式集成 LiteRT 与 AML Delegate，适用于嵌入式 Linux、方案验证与 Native 服务场景。

Interpreter 链路概览：

应用程序 / aml_delegate_test_bin → LiteRT Interpreter (C API) → AML Delegate → ADLA NPU

4.2 支持情况

当前支持的设备、算子与模型类型与 Android侧 一致：

- **设备**：搭载 ADLA NPU 的 Amlogic 平台，Linux BSP 中 ADLA 驱动与依赖库可用。
- **算子 / 数据类型**：见 2.3.2；平台精度差异见《06_Amlogic_NPU_Android_Deployment_Guide.pdf》

4.3 Interpreter API 部署

4.3.1 Interpreter C API接口说明

Linux 侧 **主接口** 为 **Interpreter C API**，职责与 Android 的 `Interpreter` + `Am1Delegate` 对应：

能力	说明
加载模型	从APK assert路径 加载 <code>.tflite</code> （Float32 / Float16 / Int8 / UInt8 等）
挂载 Delegate	注册 AML Delegate，将可委托子图下放至 NPU；不支持的算子由 LiteRT 回退至 CPU 执行
执行推理	准备输入/输出 tensor buffer，调用 run 类接口
模型路径 / 缓存	支持绝对路径；若启用编译缓存，需与模型 名称/MD5 等标识一致

Interpreter 常见交付物：

类型	示例	作用
运行时库	<code>libLiteRt.so</code>	LiteRT Interpreter 运行时库
Delegate 库	AML Delegate 配套 <code>.so</code> （名称随版本发布）	NPU 委托实现
本地 case	<code>aml_delegate_test_bin</code>	Interpreter 链路冒烟测试、精度与性能验证

4.3.2 部署流程

• **准备工作：**

确认本地已具备与目标 BSP 版本匹配的以下文件

- `libLiteRt.so` 及 AML Delegate 相关 `.so`
- `aml_delegate_test_bin`（Interpreter 验证用例）
- 测试用 `.tflite` 模型（浮点或量化均可）

• **拷贝至板端目录**

将 so 与可执行文件拷贝到板端统一工作目录（示例使用 `/data/LiteRT/`，可按项目约定调整）：

```
scp libLiteRt.so root@<board>:/data/LiteRT/
scp aml_delegate_test_bin root@<board>:/data/LiteRT/
scp <model>.tflite root@<board>:/data/LiteRT/

#在板端赋予可执行权限
chmod +x /data/LiteRT/aml_delegate_test_bin
```

• **运行 Interpreter 本地用例**

```
cd /data/LiteRT
./aml_delegate_test_bin *.tflite
```

具体参数以发布用例的说明为准（常见参数包括模型路径、输入尺寸、循环次数等）

4.3.3 应用工程集成（C/C++）

在自有 Native 工程中按 **Interpreter** 路径集成时，一般步骤如下：

- 1. 链接 `libLiteRt.so` 及 AML Delegate 相关库；
- 2. 通过 **Interpreter C API** 加载 `.tflite` 并注册 AML Delegate；
- 3. 准备输入/输出 buffer，调用推理接口；
- 4. 按需配置模型绝对路径、输入数据来源以及编译缓存目录。

4.3.4 验证建议

- 1. **初始化**：跑 `aml_delegate_test_bin`，确认 NPU 路径可用；
- 2. **精度**：对比 CPU 回退与 NPU 输出（可与 PC 金标准或 benchmark 脚本对照）；
- 3. **性能**：区分 pipeline（加载/首次编译）与 runtime 纯推理时间；
- 4. **稳定性**：长时间循环、多模型切换、异常断电恢复等。

4.4 CompiledModel API 部署(TODO)

仅在需要 CompiledModel 时使用；运行时通常需要额外动态库，与 Interpreter 主路径不同。

类型	示例	作用
Dispatch	<code>libLiteRtDispatch_Aml.so</code>	CompiledModel 运行时调度
Compiler Plugin	<code>libLiteRtCompilerPlugin_Aml.so</code>	编译/图构建插件
本地 case	<code>aml_CompiledModel_benchmark</code>	CompiledModel 链路验证